



Introduction to Cognitive Science

HomeWork #2



Sina Pirmoradian : 810101125

HW #2 Part #1

Connect Google Drive

```
1 from google.colab import drive
2 drive.mount('/content/drive')

Mounted at /content/drive
```

Import Required Library

```
1 import numpy as np
2 from scipy import signal
3 import scipy.io as sio
4 import matplotlib.pyplot as plt
5 from scipy.signal import find_peaks
6 from sklearn.decomposition import PCA
7 from sklearn.cluster import KMeans
8 from sklearn.metrics import silhouette_score
9 from sklearn.manifold import TSNE
```

Loading Data

```
1 # Load the data from the .mat file
2 data = sio.loadmat('/content/drive/MyDrive/Cognitive/HW_2/extracellular.mat')
3
4 spike = sio.loadmat('/content/drive/MyDrive/Cognitive/HW_2/spikes.mat')
5 spike_idx = spike['SpikeInds'][0]
```

Filtering: Applying a Bandpass Filter

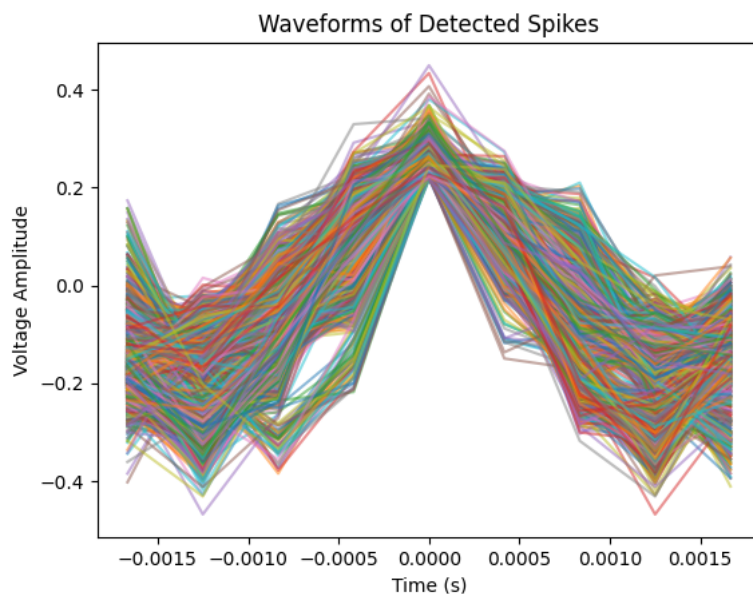
```
1 # extract the voltage amplitude data from the structure
2 voltage = data['all_data_with_noise_and_line'][0,:]
3
4 # Define filter parameters
5 fs = 2400 # Sampling rate
6 highpass_freq = 300
7 filter_order = 7
8
9 # Calculate the Nyquist frequency
10 sampling_rate = 2400
11 nyquist_freq = sampling_rate / 2
12
13 # Calculate the highpass cutoff frequency as a fraction of the Nyquist frequency
14 highpass_cutoff = highpass_freq / nyquist_freq
15
16 # Design the highpass Butterworth filter
17 b, a = signal.butter(filter_order, highpass_cutoff, btype='highpass')
18
19 # Apply the filter to the data using the filtfilt function
20 filtered_voltage = signal.filtfilt(b, a, voltage)
21
22 # Calculate the voltage threshold (0)
23 sigma_n = np.median(np.abs(filtered_voltage)) / 0.6745
24 threshold = 5 * sigma_n
25
```

```

26 print("Voltage Threshold (θ):", threshold)
27
28 # Extract peaks in the data
29 peaks, _ = find_peaks(filtered_voltage, height=0) # Find peaks with positive height
30
31 # Select points above the threshold
32 spike_points = peaks[filtered_voltage[peaks] >= threshold]
33
34 # Define the time window for waveform extraction
35 window_size = int(fs * 0.002) # 2ms before and after the peak
36 time_window = np.arange(-window_size, window_size + 1) / fs
37
38 # Extract waveforms for each detected spike
39 waveforms = []
40 for point in spike_points:
41     waveform = filtered_voltage[point - window_size: point + window_size + 1]
42     waveforms.append(waveform)
43
44 waveforms = np.array(waveforms)
45
46 # Plot the waveforms for each detected spike
47 plt.figure()
48 for waveform in waveforms:
49     plt.plot(time_window, waveform, alpha=0.5)
50
51 plt.xlabel('Time (s)')
52 plt.ylabel('Voltage Amplitude')
53 plt.title('Waveforms of Detected Spikes')
54 plt.show()
55
56 # Store the waveforms in a 2D matrix for the next step
57 # waveforms is the 2D matrix containing the waveforms
58

```

🔗 Voltage Threshold (θ): 0.2165574093077027



Spike Detection

```

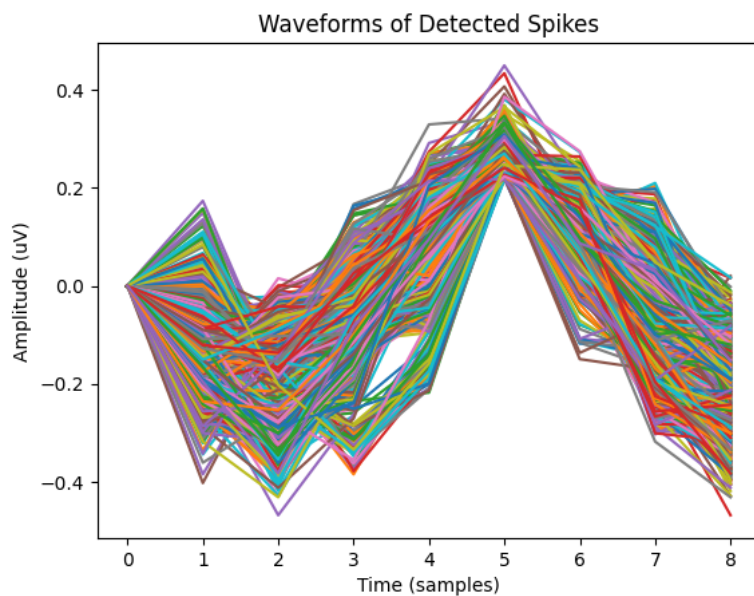
1 # Define high-pass filter parameters
2 fs = 2400 # Sampling rate
3 f_high = 300 # High-pass cutoff frequency
4 order = 7 # Filter order
5
6 # Design the filter coefficients
7 b, a = signal.butter(order, f_high / (fs / 2), btype='highpass')
8
9 # Apply the filter to the data
10 filtered_data = signal.filtfilt(b, a, voltage)
11
12 # Calculate the voltage threshold
13 sigma_n = np.median(np.abs(filtered_data)) / 0.6745
14 theta = 5 * sigma_n
15
16 # Find peaks in the data
17 diff_data = np.diff(filtered_data)
18 peaks = np.where((diff_data[:-1] > 0) & (diff_data[1:] < 0))[0] + 1

```

```

19
20 # Find spikes based on threshold
21 spikes = peaks[filtered_data[peaks] >= theta]
22
23 # Extract waveform for each spike
24 window_size = int(0.004 * fs) # 2ms window
25 waveforms = np.zeros((len(spikes), window_size))
26 for i, s in enumerate(spikes):
27     waveform_start = max(s - window_size // 2, 0)
28     waveform_end = min(s + window_size // 2, len(filtered_data))
29     waveform = filtered_data[waveform_start:waveform_end]
30     if len(waveform) < window_size:
31         waveform = np.concatenate((np.zeros(window_size - len(waveform)), waveform))
32     waveforms[i, :] = waveform
33
34 # Plot waveforms
35 plt.figure()
36 plt.plot(waveforms.T)
37 plt.xlabel('Time (samples)')
38 plt.ylabel('Amplitude (uV)')
39 plt.title('Waveforms of Detected Spikes')
40 plt.show()
41 waveforms = np.array(waveforms)

```



Feature extraction using PCA

```

1 # Apply PCA on the waveform matrix
2 pca = PCA(n_components=8)
3 waveform_pca = pca.fit_transform(waveforms)
4 print(pca.singular_values_)
5
6 PC1 = waveform_pca[:, 0]
7 PC2 = waveform_pca[:, 1]
8 PC3 = waveform_pca[:, 2]
9 PC = [PC1, PC2, PC3]
10
11 pca_waveform = PC
12
13 pca_waveform = np.array(PC)
14 pca_waveform = np.transpose(pca_waveform)

```

[8.5529941 5.61293533 3.28665284 2.82368678 2.57025087 2.49363466
1.84777402 0.72617792]

Clustering using K-Means Clustering using PCA

```

1 # Perform K-Means clustering
2 n_clusters = 3 # Number of clusters
3 kmeans = KMeans(n_clusters=n_clusters, n_init="auto")
4 kmeans.fit(pca_waveform) # Use PC1, PC2, and PC3 as features for clustering
5
6 # Get the cluster labels for each waveform
7 cluster_labels = kmeans.labels_

```

```

8
9 # Print the cluster labels
10 print("Cluster Labels:", cluster_labels)

Cluster Labels: [0 1 0 ... 1 0 1]

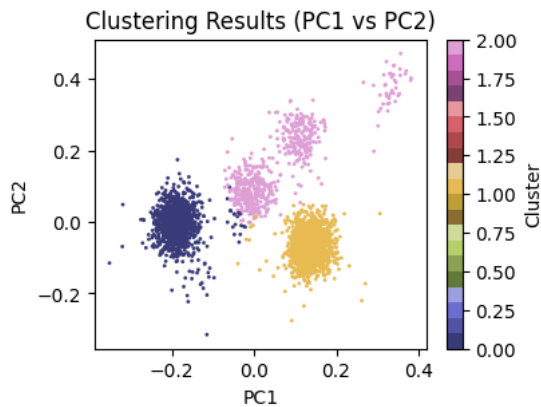
```

Plot Scatter plot of #3 main component principle

```

1 # Scatter plot for PC1 and PC2
2 plt.figure(figsize=(4, 3))
3 plt.scatter(PC1, PC2, c=cluster_labels, cmap='tab20b', s=1)
4 plt.xlabel('PC1')
5 plt.ylabel('PC2')
6 plt.title('Clustering Results (PC1 vs PC2)')
7 plt.colorbar(label='Cluster')
8 plt.show()

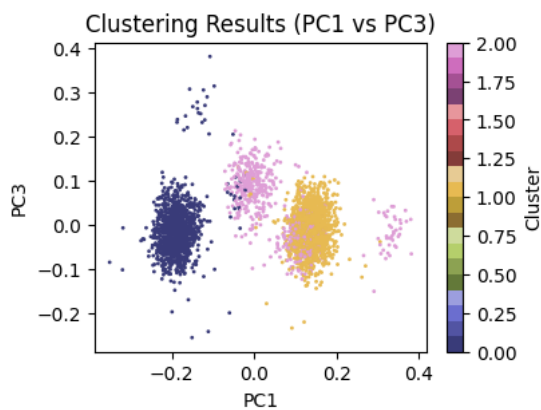
```



```

1 # Scatter plot for PC1 and PC3
2 plt.figure(figsize=(4, 3))
3 plt.scatter(PC1, PC3, c=cluster_labels, cmap='tab20b', s=1)
4 plt.xlabel('PC1')
5 plt.ylabel('PC3')
6 plt.title('Clustering Results (PC1 vs PC3)')
7 plt.colorbar(label='Cluster')
8 plt.show()

```



Using silhouette_score to Find a K which was the best performance using PCA

```

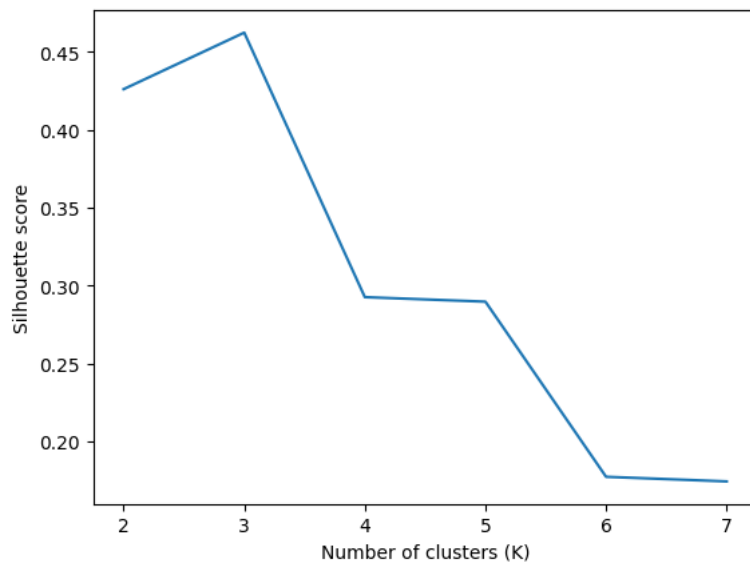
1 # Define a range of values for K
2 k_values = range(2, 8)
3
4 # Calculate silhouette scores for each value of K
5 silhouette_scores = []
6 for k in k_values:
7     kmeans = KMeans(n_clusters=k, random_state=10, n_init="auto")
8     labels = kmeans.fit_predict(waveform_pca)
9     score = silhouette_score(waveform_pca, labels)
10    silhouette_scores.append(score)
11
12 # Plot the silhouette scores for each value of K
13 import matplotlib.pyplot as plt
14

```

```

15 plt.plot(k_values, silhouette_scores)
16 plt.xlabel('Number of clusters (K)')
17 plt.ylabel('Silhouette score')
18 plt.show()
19

```

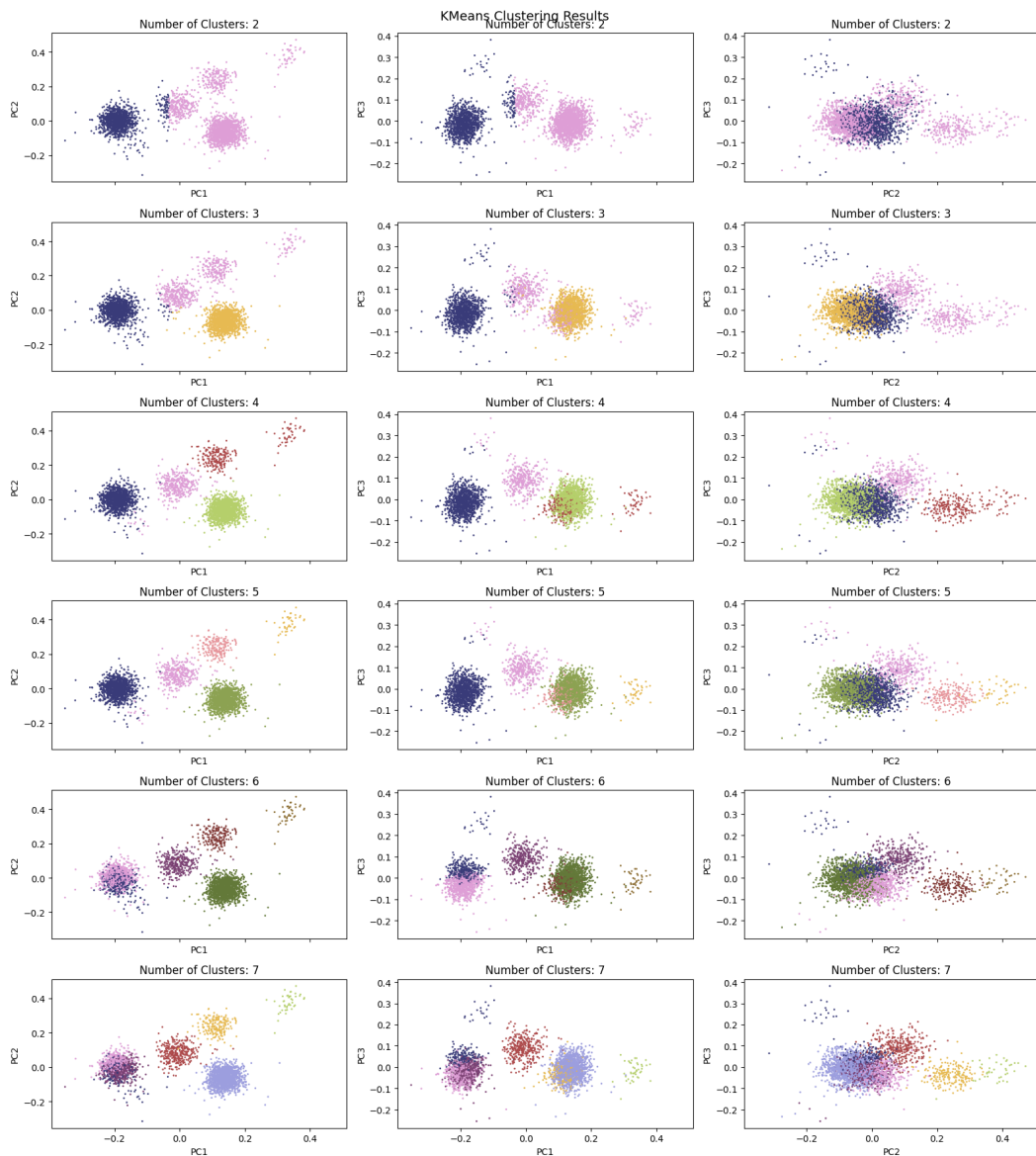


Plot the Result of clustering with different K's using PCA

```

1 # Repeat part (k) and (l) with different values of K for the clustering algorithm
2 k = 8
3 k_values = range(2, k)
4 fig, axes = plt.subplots(k-2, 3, sharex=True, figsize=(4 * 4, (k-2) * 3))
5 for i in k_values:
6     kmeans = KMeans(n_clusters=i, random_state=k, n_init="auto").fit(waveform_pca)
7     labels = kmeans.labels_
8     axes[i-2, 0].scatter(waveform_pca[:,0], waveform_pca[:,1], c = labels, s=1, cmap='tab20b')
9     axes[i-2, 0].set_title('Number of Clusters: ' + str(i))
10    axes[i-2, 0].set_xlabel('PC1')
11    axes[i-2, 0].set_ylabel('PC2')
12
13    axes[i-2, 1].scatter(waveform_pca[:,0], waveform_pca[:,2], c = labels, s=1, cmap='tab20b')
14    axes[i-2, 1].set_title('Number of Clusters: ' + str(i))
15    axes[i-2, 1].set_xlabel('PC1')
16    axes[i-2, 1].set_ylabel('PC3')
17
18    axes[i-2, 2].scatter(waveform_pca[:,1], waveform_pca[:,2], c = labels, s=1, cmap='tab20b')
19    axes[i-2, 2].set_title('Number of Clusters: ' + str(i))
20    axes[i-2, 2].set_xlabel('PC2')
21    axes[i-2, 2].set_ylabel('PC3')
22
23 # Add overall title and adjust layout
24 fig.suptitle('KMeans Clustering Results', fontsize=14)
25 fig.tight_layout()

```



Compute performance metrics of the K-means clustering by using PCA

```

1 # Compute performance metrics
2 time_window = 0.001 # +/- 1ms
3 true_positives = 0
4 false_positives = 0
5 false_negatives = 0
6
7 for spike_time in spike_idx:
8     if np.any(np.abs(peaks - spike_time) <= time_window * fs):
9         true_positives += 1
10    else:
11        false_negatives += 1
12
13 for spike_time in spikes:
14     if np.all(np.abs(spike_time - spike_idx) > time_window * fs):
15         false_positives += 1
16
17 precision = true_positives / (true_positives + false_positives)
18 recall = true_positives / (true_positives + false_negatives)

```

```

19 f1_score = 2 * (precision * recall) / (precision + recall)
20
21 print("Precision: %.3f" % precision)
22 print("Recall: %.3f" % recall)
23 print("F1-Score: %.3f" % f1_score)
24
    Precision: 0.772
    Recall: 0.957
    F1-Score: 0.855

```

Filtering: Applying a Bandpass Filter Using New Threshold

```

1 # extract the voltage amplitude data from the structure
2 voltage = data['all_data_with_noise_and_line'][0,:]
3
4 # Define high-pass filter parameters
5 fs = 2400 # Sampling rate
6 f_high = 300 # High-pass cutoff frequency
7 order = 7 # Filter order
8
9 # Design the filter coefficients
10 b, a = signal.butter(order, f_high / (fs / 2), btype='highpass')
11
12 # Apply the filter to the data
13 filtered_data = signal.filtfilt(b, a, voltage)
14
15 # Find peaks in the data
16 diff_data = np.diff(filtered_data)
17 peaks = np.where((diff_data[:-1] > 0) & (diff_data[1:] < 0))[0] + 1
18
19 # Calculate the voltage threshold
20 new_theta = 0.8 * np.max(filtered_data)
21 p = filtered_data[peaks] >= new_theta
22
23 # Find spikes based on threshold
24 new_spikes = peaks[filtered_data[peaks] >= new_theta]
25 print(new_spikes)
26
27 # Extract waveform for each spike
28 window_size = int(0.004 * fs) # 2ms window
29 new_waveforms = np.zeros((len(new_spikes), window_size))
30 for i, s in enumerate(new_spikes):
31     new_waveform_start = max(s - window_size // 2, 0)
32     new_waveform_end = min(s + window_size // 2, len(filtered_data))
33     new_waveform = filtered_data[new_waveform_start:new_waveform_end]
34     if len(new_waveform) < window_size:
35         new_waveform = np.concatenate((np.zeros(window_size - len(new_waveform)), new_waveform))
36     new_waveforms[i, :] = new_waveform
37
38 # Plot waveforms
39 plt.figure()
40 plt.plot(new_waveforms.T)
41 plt.xlabel('Time (samples)')
42 plt.ylabel('Amplitude (uV)')
43 plt.title('Waveforms of Detected Spikes')
44 plt.show()
45 new_waveforms = np.array(new_waveforms)

```

```
[ 845516 2350824 3253884 3643715 3684329 3691450 3691456 4107918 4393995
 4403675]
```

Waveforms of Detected Spikes



Using K-Means Clustering with New Threshold



```
1 # Apply PCA on the waveform matrix
2 pca = PCA(n_components=8)
3 new_waveform_pca = pca.fit_transform(new_waveforms)
4 print(pca.singular_values_)
5
6 new_PC1 = new_waveform_pca[:, 0]
7 new_PC2 = new_waveform_pca[:, 1]
8 new_PC3 = new_waveform_pca[:, 2]
9 new_PC = [new_PC1, new_PC2, new_PC3]
10
11 new_pca_waveform = new_PC
12
13 new_pca_waveform = np.array(new_PC)
14 new_pca_waveform = np.transpose(new_pca_waveform)

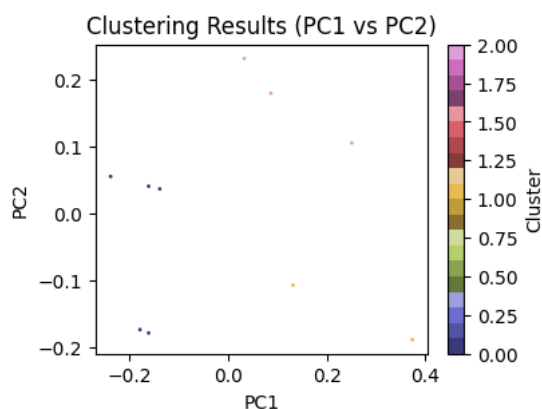
[0.62258675 0.45924231 0.25186375 0.14795406 0.0889995 0.07161821
 0.06647049 0.01176512]

1 # Perform K-Means clustering
2 n_clusters = 3 # Number of clusters
3 kmeans = KMeans(n_clusters=n_clusters, n_init="auto")
4 kmeans.fit(new_pca_waveform) # Use PC1, PC2, and PC3 as features for clustering
5
6 # Get the cluster labels for each waveform
7 new_cluster_labels = kmeans.labels_
8
9 # Print the cluster labels
10 print("Cluster Labels:", new_cluster_labels)

Cluster Labels: [0 0 0 2 0 2 0 2 1 2 1]
```

Scatter plot of #3 main component principle using new threshold

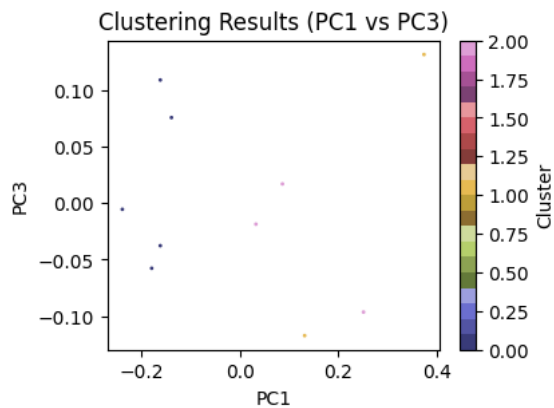
```
1 import matplotlib.pyplot as plt
2
3 # Scatter plot for PC1 and PC2
4 plt.figure(figsize=(4, 3))
5 plt.scatter(new_PC1, new_PC2, c=new_cluster_labels, cmap='tab20b', s=1)
6 plt.xlabel('PC1')
7 plt.ylabel('PC2')
8 plt.title('Clustering Results (PC1 vs PC2)')
9 plt.colorbar(label='Cluster')
10 plt.show()
```




```

1 # Scatter plot for PC1 and PC3
2 plt.figure(figsize=(4, 3))
3 plt.scatter(new_PC1, new_PC3, c=new_cluster_labels, cmap='tab20b', s=1)
4 plt.xlabel('PC1')
5 plt.ylabel('PC3')
6 plt.title('Clustering Results (PC1 vs PC3)')
7 plt.colorbar(label='Cluster')
8 plt.show()

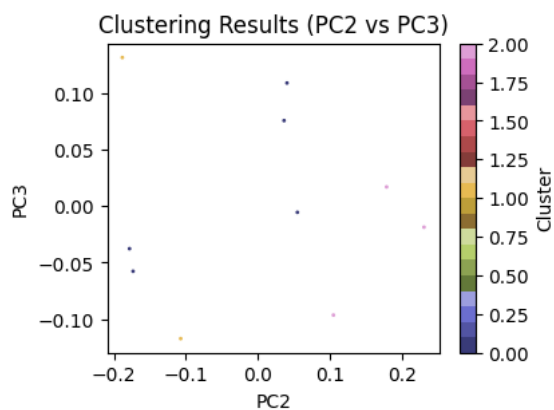
```



```

1 # Scatter plot for PC2 and PC3
2 plt.figure(figsize=(4, 3))
3 plt.scatter(new_PC2, new_PC3, c=new_cluster_labels, cmap='tab20b', s=1)
4 plt.xlabel('PC2')
5 plt.ylabel('PC3')
6 plt.title('Clustering Results (PC2 vs PC3)')
7 plt.colorbar(label='Cluster')
8 plt.show()

```

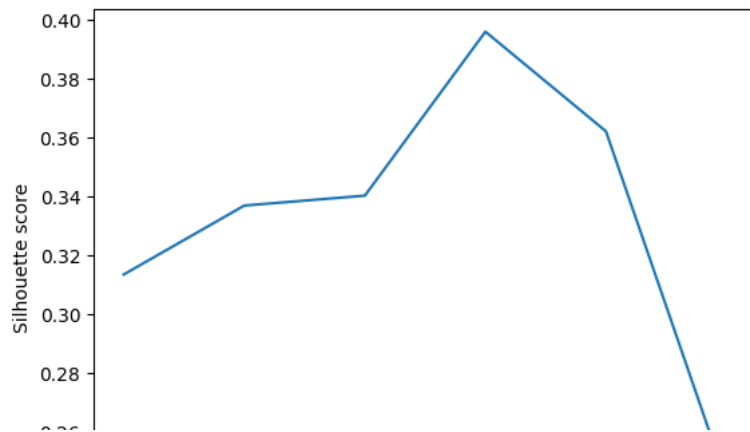


Using silhouette_score to Find a K which was the best performance using new threshold

```

1 # Define a range of values for K
2 k_values = range(2, 8)
3
4 # Calculate silhouette scores for each value of K
5 new_silhouette_scores = []
6 for k in k_values:
7     kmeans = KMeans(n_clusters=k, random_state=10, n_init="auto")
8     new_labels = kmeans.fit_predict(new_waveform_pca)
9     score = silhouette_score(new_waveform_pca, new_labels)
10    new_silhouette_scores.append(score)
11
12 # Plot the silhouette scores for each value of K
13 plt.plot(k_values, new_silhouette_scores)
14 plt.xlabel('Number of clusters (K)')
15 plt.ylabel('Silhouette score')
16 plt.show()
17

```

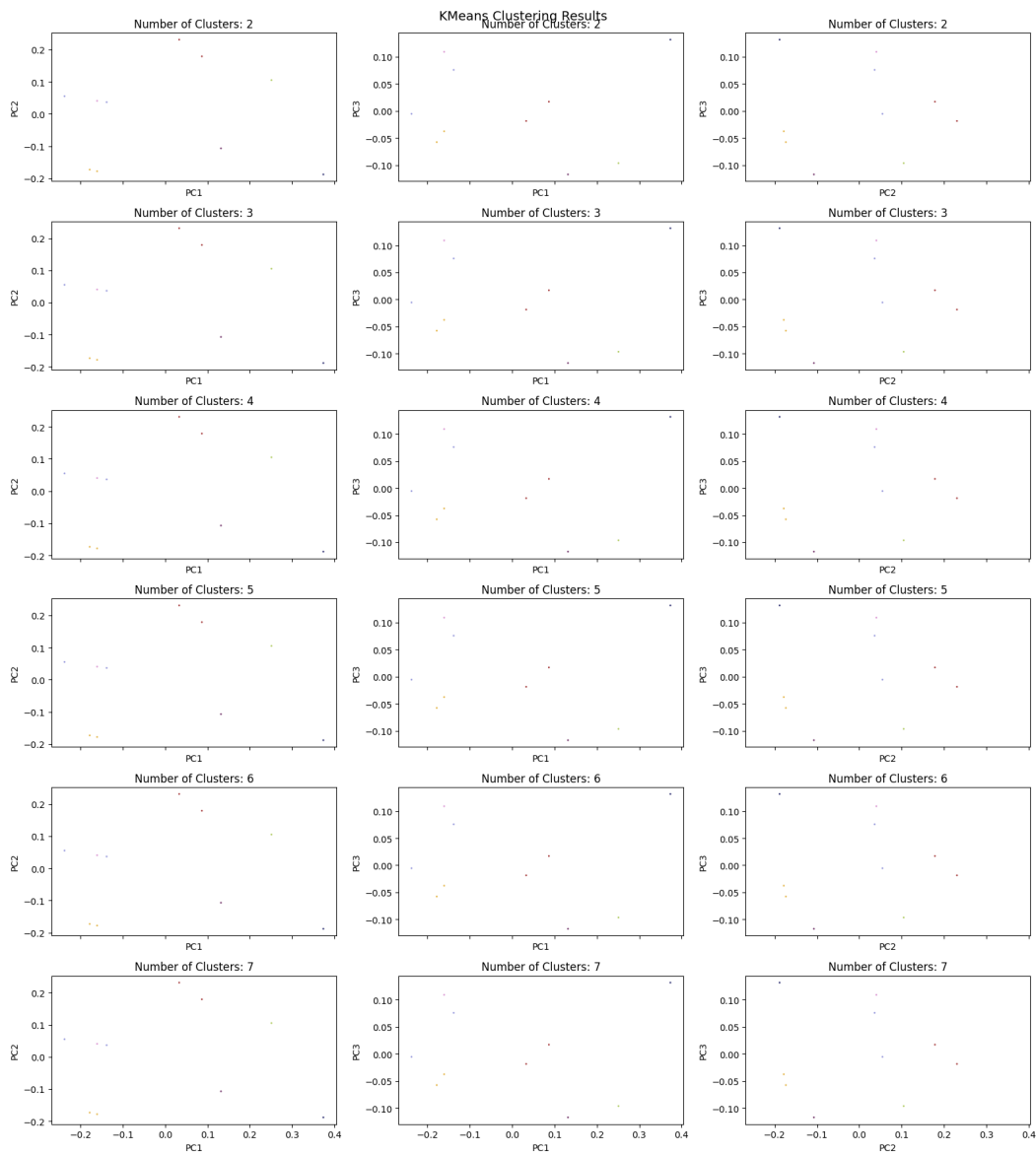


Plot the Result of clustering with different K's with new threshold

```

1 # Repeat part (k) and (l) with different values of K for the clustering algorithm
2 k = 8
3 k_values = range(2, k)
4 fig, axes = plt.subplots(k-2, 3, sharex=True, figsize=(4 * 4, (k-2) * 3))
5 for i in k_values:
6     kmeans = KMeans(n_clusters=i, random_state=k, n_init="auto").fit(new_waveform_pca)
7     labels = kmeans.labels_
8     axes[i-2, 0].scatter(new_waveform_pca[:,0], new_waveform_pca[:,1], c = new_labels, s=1, cmap='tab20b')
9     axes[i-2, 0].set_title('Number of Clusters: ' + str(i))
10    axes[i-2, 0].set_xlabel('PC1')
11    axes[i-2, 0].set_ylabel('PC2')
12
13    axes[i-2, 1].scatter(new_waveform_pca[:,0], new_waveform_pca[:,2], c = new_labels, s=1, cmap='tab20b')
14    axes[i-2, 1].set_title('Number of Clusters: ' + str(i))
15    axes[i-2, 1].set_xlabel('PC1')
16    axes[i-2, 1].set_ylabel('PC3')
17
18    axes[i-2, 2].scatter(new_waveform_pca[:,1], new_waveform_pca[:,2], c = new_labels, s=1, cmap='tab20b')
19    axes[i-2, 2].set_title('Number of Clusters: ' + str(i))
20    axes[i-2, 2].set_xlabel('PC2')
21    axes[i-2, 2].set_ylabel('PC3')
22
23 # Add overall title and adjust layout
24 fig.suptitle('KMeans Clustering Results', fontsize=14)
25 fig.tight_layout()

```



Compute performance metrics of the K-means clustering for new threshold

```

1 # Compute performance metrics
2 time_window = 0.001 # +/- 1ms
3 true_positives = 0
4 false_positives = 0
5 false_negatives = 0
6
7 for spike_time in spike_idx:
8     if np.any(np.abs(peaks - spike_time) <= time_window * fs):
9         true_positives += 1
10    else:
11        false_negatives += 1
12
13 for spike_time in new_spikes:
14     if np.all(np.abs(spike_time - spike_idx) > time_window * fs):
15         false_positives += 1
16
17 precision = true_positives / (true_positives + false_positives)
18 recall = true_positives / (true_positives + false_negatives)
19 f1_score = 2 * (precision * recall) / (precision + recall)
20
21 print("Precision: %.3f" % precision)
22 print("Recall: %.3f" % recall)
23 print("F1-Score: %.3f" % f1_score)
24

```

```

Precision: 0.999
Recall: 0.957
F1-Score: 0.978

```

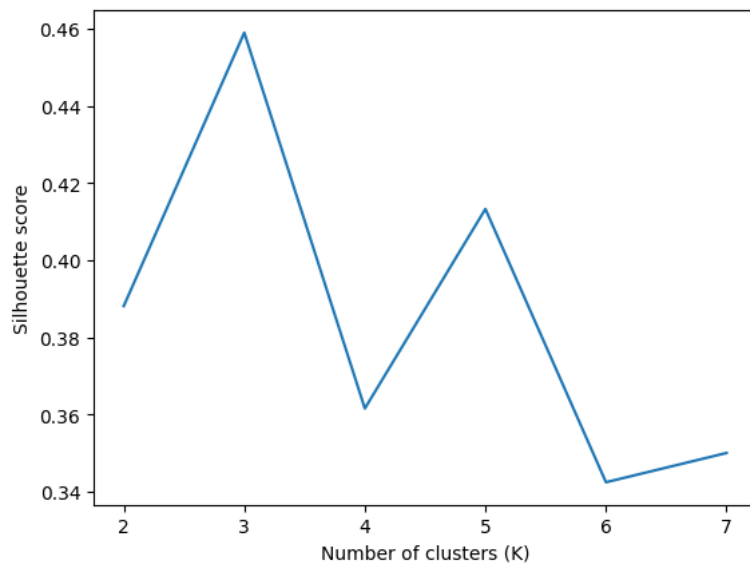
Feature extraction and Dimension Reduction Using t-SNE

```
1 # Apply t-SNE to reduce dimensionality
2 tsne = TSNE(n_components=3, random_state=0)
3 tsne_feature = tsne.fit_transform(waveforms)

1 tsne_1 = tsne_feature[:, 0]
2 tsne_2 = tsne_feature[:, 1]
3 tsne_3 = tsne_feature[:, 2]
4 tsne_features = [tsne_1, tsne_2, tsne_3]
5
6 tsne_features = np.array(tsne_features)
7 tsne_features = np.transpose(tsne_features)
```

Using silhouette_score to Find a K which was the best performance for t-SNE algorithm

```
1 # Define a range of values for K
2 k_values = range(2, 8)
3
4 # Calculate silhouette scores for each value of K
5 silhouette_scores = []
6 for k in k_values:
7     kmeans = KMeans(n_clusters=k, random_state=10, n_init="auto")
8     labels = kmeans.fit_predict(tsne_features)
9     score = silhouette_score(tsne_features, labels)
10    silhouette_scores.append(score)
11
12 # Plot the silhouette scores for each value of K
13 plt.plot(k_values, silhouette_scores)
14 plt.xlabel('Number of clusters (K)')
15 plt.ylabel('Silhouette score')
16 plt.show()
17
```



Plot the Result of clustering with different K's using t-SNE algorithm

```
1 # Repeat part (k) and (l) with different values of K for the clustering algorithm
2 k = 8
3 k_values = range(2, k)
4 fig, axes = plt.subplots(k-2, 3, sharex=True, figsize=(4 * 4, (k-2) * 3))
5 for i in k_values:
6     kmeans = KMeans(n_clusters=i, random_state=k, n_init="auto").fit(tsne_features)
7     labels = kmeans.labels_
8     axes[i-2, 0].scatter(tsne_feature[:,0], tsne_features[:,1], c = labels, s=1, cmap='tab20b')
9     axes[i-2, 0].set_title('Number of Clusters: ' + str(i))
10
11    axes[i-2, 1].scatter(tsne_feature[:,0], tsne_features[:,2], c = labels, s=1, cmap='tab20b')
12    axes[i-2, 1].set_title('Number of Clusters: ' + str(i))
13
14    axes[i-2, 2].scatter(tsne_feature[:,1], tsne_features[:,2], c = labels, s=1, cmap='tab20b')
15    axes[i-2, 2].set_title('Number of Clusters: ' + str(i))
16
17
```

```
18 # Add overall title and adjust layout
19 fig.suptitle('KMeans Clustering Results', fontsize=14)
20 fig.tight_layout()
```

